

SBA Report

by Lo Sui Nam (6A)

Introduction of the task

Automatically generate a competition chart that satisfies the following:

- Separate players from the same school where possible
- Separate seed players and avoid early meetups
- Use 'byes' only in the first round and balance them
- Keep generating the next round until there is a champion
- Randomize the result of each match, with seeded players having an advantage
- Optional task: create a player enquiry function for user to check the player's record

Libraries used in the program

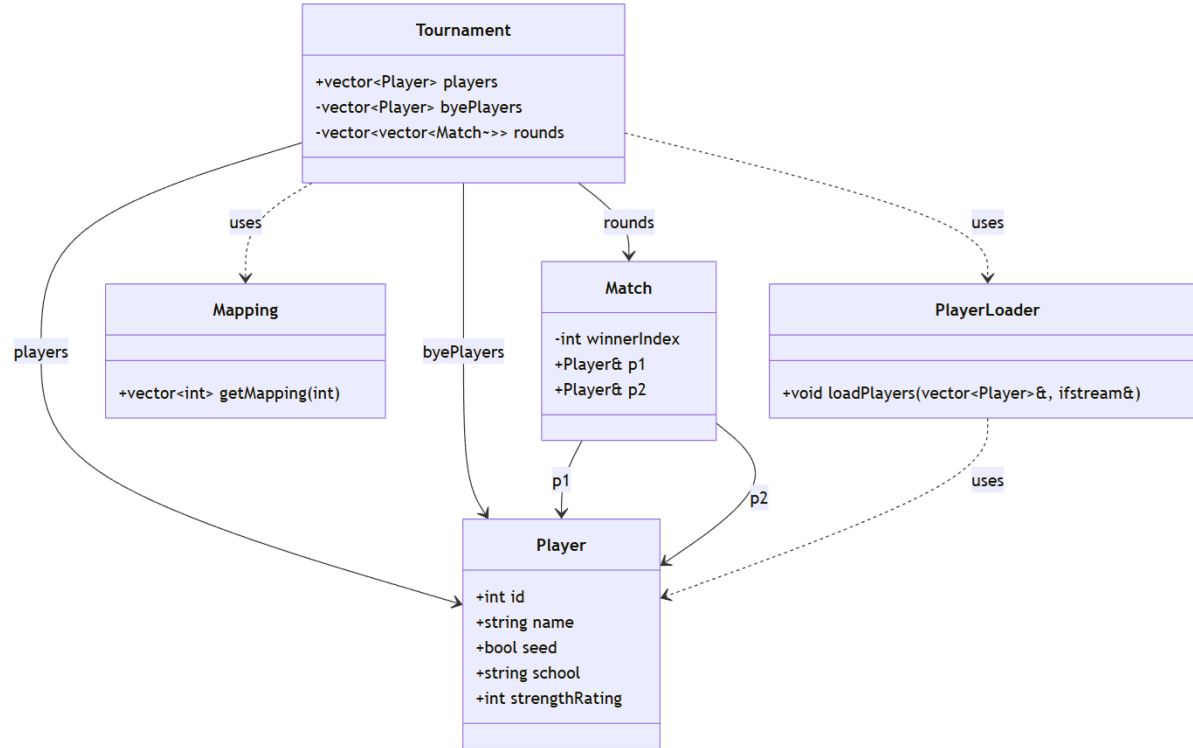
iostream	to use the basic <code>cin cout</code> functions to handle user input and output
fstream	to handle file input and output
sstream	provides stringstream class that is capable of writing different datatypes into a single stream of text
iomanip	provides a set of manipulators that are used to format the output
vector	to use dynamic array which its size can be changed in runtime, and provides the <code>push_back</code> function
string	to use the datatype string
math.h	provides a series of functions for mathematical operations
algorithm	to use the <code>shuffle</code> function to randomly reorder elements in a vector
random	the <code>shuffle</code> function requires this library for random outcomes

Classes

Classes used in the program

- This program consists of 3 main classes (Player, Match, Tournament) and 2 helper classes (Mapping, PlayerLoader)

Class diagram



class: Player

```
8  class Player
9  {
10 public:
11     int id;
12     string name;
13     bool seed;
14     string school;
15
16     int strengthRating;
17
18     //Default constructor - represents a "Bye" (Empty slot)
19     Player()
20     {
21         name = "Bye";
22         seed = false;
23         school = "";
24
25         strengthRating = 0;
26     }
27
28     //constructor - real competitor
29     Player(string _name, bool _seed, string _school)
30     {
31         name = _name;
32         seed = _seed;
33         school = _school;
34         if (seed)
35             strengthRating = 70;
36         if (!seed)
37             strengthRating = 30;
38     }
39 };
40
```

Member variables:

<i>Variable</i>	<i>Data Type</i>	<i>Description</i>
id	int	a unique id assigned to each player automatically, used for tracking
name	string	the name of the player (or "Bye" in case of empty slot)
seed	bool	whether the player is seeded or not
school	string	the school name of the player
strengthRating	int	a relative strength assigned to the player (Seeded = 70, Unseeded = 30, "Bye" = 0)

class: Match

```
10 class Match
11 {
12 private:
13     int winnerIndex;
14
15 public:
16     Player &p1;
17     Player &p2;
18
19     // Constructor: takes 2 Players by reference
20     Match(Player &p1, Player &p2) : p1(_p1), p2(_p2) {};
21
22 > Player &simulateMatch() ...
36
37 > string getMatchResult(){ ...
54
55 > Player &getWinner(){ ...
58
59
60
61 };
62
```

Overview:

- private member variable: winnerIndex
- public member variables: p1, p2
- public methods:
 - simulateMatch()
 - getMatchResult()
 - getWinner()

Match - Member variables

```
10 class Match
11 {
12 private:
13     int winnerIndex;
14
15 public:
16     Player &p1;
17     Player &p2;
18
19     // Constructor: takes 2 Players by reference
20     Match(Player &p1, Player &p2) : p1(_p1), p2(_p2) {};
21
22 > Player &simulateMatch() ...
36
37 > string getMatchResult(){ ...
54
55 > Player &getWinner(){ ...
58
59
60
61 };
62
```

<i>Variable</i>	<i>Data Type</i>	<i>Description</i>
p1	reference of Player	stores the reference to the first competitor in the match
p2	reference of Player	stores the reference to the second competitor in the match
winnerIndex	int	stores the index of winning player after simulateMatch() is ran (0 for p1, 1 for p2)

* The two competitors are stored as **references** to avoid duplicating player data, so any updates to players' properties will be applied to the **original** object

Public method - simulate()

-> Returns reference to the winner

```
22  Player &simulateMatch()
23  {
24      int randomNum = rand() % (p1.strengthRating + p2.strengthRating) + 1;
25      if (randomNum <= p1.strengthRating){
26          winnerIndex = 0;
27          return p1;
28      }
29      else{
30          winnerIndex = 1;
31          return p2;
32      }
33  }
```

This method determines the winner of the match using a weighted random algorithm based on each player's **strengthRating**

- Generates random number from 1 to ($p1 + p2$ strength)
- $p1$ wins if number $\leq p1.\text{strengthRating}$, vice versa
- Returns reference to winning player
- Set the **winnerIndex** for later use

For example:

- $p1$ has a strength of 30, $p2$ has a strength of 70
- the number generated is 56
- 56 is greater than 30
- $p2$ wins

Public method - getMatchResult()

-> Returns a string

```
35  ✓  string getMatchResult(){
36      stringstream ss;
37      ss<<p1.name;
38      ss<<(p1.seed ? "*" : "");
39
40      ss<<" vs ";
41      ss<<p2.name;
42      ss<<(p2.seed ? "*" : "");
43
44      ss<<endl;
45
46      ss<<"Match winner: ";
47      ss<<getWinner().name;
48      ss<<(getWinner().seed ? "*" : "")<<endl;
49
50      return ss.str();
51  }
```

This method returns a formatted string that clearly displays:

- Both players' names
- A visual indicator if a player is seeded (shows * after the name)
- Who won the match

an example output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Kyle\Desktop\C (1)> g++ main.cpp
PS C:\Users\Kyle\Desktop\C (1)> ./a.exe

Kathy Yip* vs Eddie Hui
Match winner: Kathy Yip*

PS C:\Users\Kyle\Desktop\C (1)> |
```

Public method - getWinner()

-> Returns reference to winner

Returns the winning player based on **winnerIndex**

- Returns *p1* if **winnerIndex** is 0
- Returns *p2* if **winnerIndex** is 1

*This does not re-run the match simulation

*Only returns the already-decided winner for later use (eg. printing outcomes)

```
55 Player &getWinner(){  
56     return winnerIndex == 0 ? p1 : p2;  
57 }
```

class: Tournament

```
66
67 class Tournament
68 {
69 public:
70     vector<Player> players;
71
72 >     stringstream generatePlayerReport(int id){...
98
99 >     stringstream generateTournamentReport(){...
119
120 >     stringstream generatePlayerList(){...
141
142 >     void printTo(stringstream text, ostream& os){...
145
146 >     void simulateTournament()...
156
157 private:
158     vector<Player> byePlayers;
159     vector<vector<Match>> rounds;
160
161 >     void createByePlayers(int numberOfByePlayers) ...
168
169 >     int getNumberOfMatches(int playerListSize) ...
178
179 >     void createInitialMatches() ...
235
236 >     vector<Match> rearrangeMatches(vector<Match> &matches) ...
253
254 >     void proceedNextRound(vector<Match> &matches) ...
276
277 >     string getRoundName(int numberOfPlayers) ...
289
290 >     vector<reference_wrapper<Player>> groupSeededPlayers() ...
302 >     vector<reference_wrapper<Player>> groupUnseededPlayers
314 >     () ...
315 >     vector<reference_wrapper<Player>> mergeAndFillPlayers
344 >     (vector<reference_wrapper<Player>> &seededPlayers,
345 >     vector<reference_wrapper<Player>> &unseededPlayers) ...
346 > };
```

Overview:

- private member variable: byePlayers, rounds
- public member variables: players
- 4 public methods for printing result
- 8 private methods for building the tournament
- 1 public method for running the simulation

Tournament - Member variables

```
66
67 class Tournament
68 {
69 public:
70     vector<Player> players;
71
72 >     stringstream generatePlayerReport(int id){...
98
99 >     stringstream generateTournamentReport(){...
119
120 >     stringstream generatePlayerList(){...
141
142 >     void printTo(stringstream text, ostream& os){...
145
146 >     void simulateTournament() ...
156
157 private:
158     vector<Player> byePlayers;
159     vector<vector<Match>> rounds;
160
161 >     void createByePlayers(int numberOfByePlayers) ...
168
169 >     int getNumberOfMatches(int playerListSize) ...
178
179 >     void createInitialMatches() ...
235
236 >     vector<Match> rearrangeMatches(vector<Match> &matches) ...
253
254 >     void proceedNextRound(vector<Match> &matches) ...
276
277 >     string getRoundName(int numberOfPlayers) ...
289
290 >     vector<reference_wrapper<Player>> groupSeededPlayers() ...
302 >     vector<reference_wrapper<Player>> groupUnseededPlayers
    () ...
314
315 >     vector<reference_wrapper<Player>> mergeAndFillPlayers
    (vector<reference_wrapper<Player>> &seededPlayers,
    vector<reference_wrapper<Player>> &unseededPlayers) ...
344 };
345
```

Public:

<i>Variable</i>	<i>Data Type</i>	<i>Description</i>
players	vector of Player	an array of all players

Private:

<i>Variable</i>	<i>Data Type</i>	<i>Description</i>
byePlayers	vector< Player >	an array of “byes” (auto promotion)
rounds	vector<vector< Match >>	an array that stores all the matches in the tournament

Public method - getNumberOfMatches()

-> Returns int

```
168
169     int getNumberOfMatches(int playerListSize)
170     {
171         //special cases
172         if (playerListSize == 0)
173             return 0;
174         if (playerListSize == 1)
175             return 1;
176
177
178         return pow(2, ceil(log(playerListSize) / log(2))) / 2;
179     };
180
```

- Calculate how many matches are needed based on the number of players
- This uses some math functions to derive the number by finding the next closest power of two
- eg.
 - 4 players requires 2 matches
 - 16 players requires 8 matches
 - 32 players requires 16 matches
 - 20 players also requires 16 matches (with some auto-promotion)
 - 1 player requires exactly 1 match (automatically wins)

Public method - createInitialMatches()

```
180
181 void createInitialMatches()
182 {
183     vector<reference_wrapper<Player>> seededPlayers = groupSeededPlayers();
184     vector<reference_wrapper<Player>> unseededPlayers = groupUnseededPlayers();
185
186     vector<reference_wrapper<Player>> arrangedPlayers = mergeAndFillPlayers
187         (seededPlayers, unseededPlayers);
188
189     // create first round matches
190     vector<Match> firstRoundMatches;
191     int playerCount = getNumberOfMatches(players.size()) * 2 - 1;
192
193     for (int i = 0; i < arrangedPlayers.size() / 2; i++) ...
194
195     for (int i = 0; i < arrangedPlayers.size() / 2; i++) ...
196
197     // rearrange the matches orders
198     rounds.push_back(rearrangeMatches(firstRoundMatches));
199
200     return;
201 }
```

This builds the 1st round of the tournament using the following methods:

- groupSeededPlayers(), groupUnseededPlayers()
- mergeAndFillPlayers()
- rearrangeMatches()

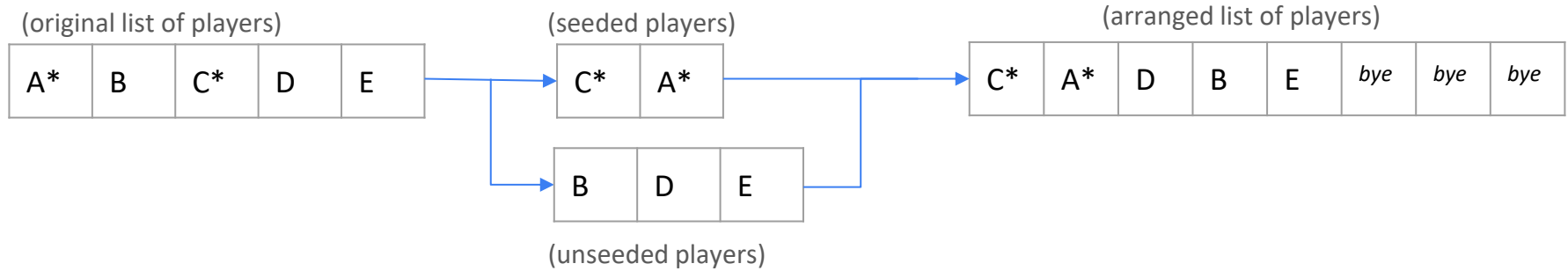
Finally, store all the matches into **rounds** for future use

Public method - createInitialMatches()

```
180
181 void createInitialMatches()
182 {
183     vector<reference_wrapper<Player>> seededPlayers = groupSeededPlayers();
184     vector<reference_wrapper<Player>> unseededPlayers = groupUnseededPlayers();
185
186     vector<reference_wrapper<Player>> arrangedPlayers = mergeAndFillPlayers
187         (seededPlayers, unseededPlayers);
188
189     // create first round matches
190     vector<Match> firstRoundMatches;
191     int playerCount = getNumberOfMatches(players.size()) * 2 - 1;
192
193     for (int i = 0; i < arrangedPlayers.size() / 2; i++) ...
194
195     for (int i = 0; i < arrangedPlayers.size() / 2; i++) ...
196
197     // rearrange the matches orders
198     rounds.push_back(rearrangeMatches(firstRoundMatches));
199
200     return;
201 }
```

1. separates seed and unseed players
2. shuffle each group individually
3. merge + add “byes”, to form an arranged queue in the order of **seed, unseeded, byes**

Visual presentation – arranging the queue



Public method - createInitialMatches()

```
180
181 void createInitialMatches()
182 {
183     vector<reference_wrapper<Player>> seededPlayers = groupSeededPlayers();
184     vector<reference_wrapper<Player>> unseededPlayers = groupUnseededPlayers();
185
186     vector<reference_wrapper<Player>> arrangedPlayers = mergeAndFillPlayers
187         (seededPlayers, unseededPlayers);
188
189     // create first round matches
190     vector<Match> firstRoundMatches;
191     int playerCount = getNumberOfMatches(players.size()) * 2 - 1;
192
193     for (int i = 0; i < arrangedPlayers.size() / 2; i++) ...
194
195     for (int i = 0; i < arrangedPlayers.size() / 2; i++) ...
196
197     // rearrange the matches orders
198     rounds.push_back(rearrangeMatches(firstRoundMatches));
199
200     return;
201 }
```

```
190
191 for (int i = 0; i < arrangedPlayers.size() / 2; i++)
192 {
193     if (arrangedPlayers[i].get().school == arrangedPlayers[playerCount - i].get().school){
194
195         if (i >= playerCount / 2){
196             // swap with back if there is no player in front of it to swap with
197             auto temp = arrangedPlayers[playerCount - i];
198             arrangedPlayers[playerCount - i] = arrangedPlayers[playerCount - i + 1];
199             arrangedPlayers[playerCount - i - 1] = temp;
200
201         }else{
202             //swap with front
203             auto temp = arrangedPlayers[playerCount - i];
204             arrangedPlayers[playerCount - i] = arrangedPlayers[playerCount - i - 1];
205             arrangedPlayers[playerCount - i - 1] = temp;
206         }
207     }
208 }
209
210
211 for (int i = 0; i < arrangedPlayers.size() / 2; i++)
212 {
213     firstRoundMatches.push_back(Match(arrangedPlayers[i], arrangedPlayers[playerCount - i]));
214 }
215 }
```

Public method - createInitialMatches()

```
190
191 for (int i = 0; i < arrangedPlayers.size() / 2; i++)
192 {
193     if (arrangedPlayers[i].get().school == arrangedPlayers[playerCount - i].get().school){
194
195         if (i >= playerCount / 2){
196             // swap with back if there is no player in front of it to swap with
197             auto temp = arrangedPlayers[playerCount - i];
198             arrangedPlayers[playerCount - i] = arrangedPlayers[playerCount - i + 1];
199             arrangedPlayers[playerCount - i - 1] = temp;
200
201         }else{
202             //swap with front
203             auto temp = arrangedPlayers[playerCount - i];
204             arrangedPlayers[playerCount - i] = arrangedPlayers[playerCount - i - 1];
205             arrangedPlayers[playerCount - i - 1] = temp;
206         }
207     }
208 }
209
210
211 for (int i = 0; i < arrangedPlayers.size() / 2; i++)
212 {
213     firstRoundMatches.push_back(Match(arrangedPlayers[i], arrangedPlayers[playerCount - i]));
214 }
215
```

Using the arranged queue to generate all 1st round matchups, while avoiding pairing seeded players or same school players together

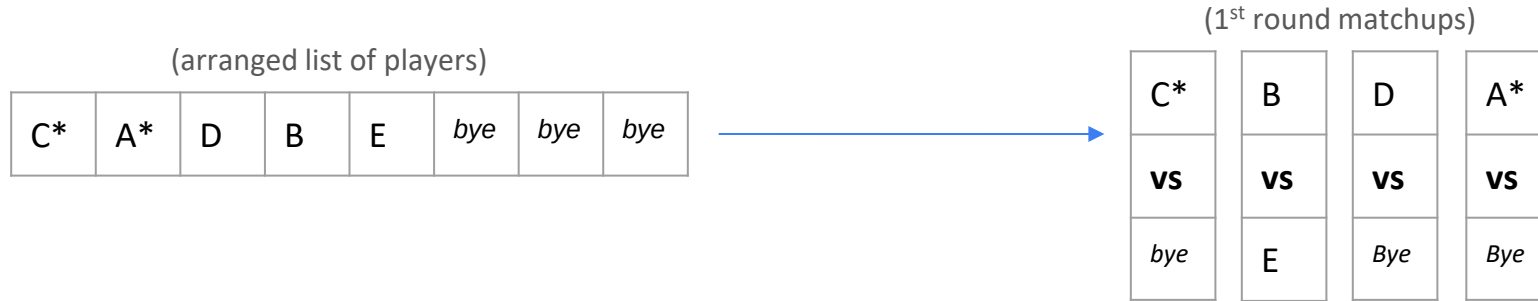
1. Pair the first and last player of the queue to form a match
2. Check if both players are from the same school
3. If yes, pair with another player
4. Repeat until all players are paired

Public method - createInitialMatches()

```
18
19 class Mapping
20 {
21 public:
22     static vector<int> getMapping(int matchCount){
23         if (matchCount == 4) return {0,2,3,1};
24         if (matchCount == 8) return {0,4,6,2,3,7,5,1};
25         if (matchCount == 16) return {0,8,12,4,6,14,10,2,3,11,15,7,5,13,9,1};
26         else return {};
27     }
28 };
29
```

Reorder the matches to separate seeded players across the bracket so they only meet in later rounds using the **Mapping** class

Visual presentation – pairing the matchups



Public method - proceedNextRound()

```
239
240 void proceedNextRound(vector<Match> &matches)
241 {
242     vector<reference_wrapper<Player>> winners;
243     vector<Match> progressedMatches;
244
245     for (Match &match : matches)
246     {
247         winners.push_back(match.simulateMatch());
248     }
249
250     if (winners.size() == 1)
251     {
252         return;
253     }
254
255     for (int i = 0; i < winners.size(); i += 2)
256     {
257         progressedMatches.push_back(Match(winners[i], winners[i + 1]));
258     }
259
260     rounds.push_back(progressedMatches);
261 }
262
```

This method promotes the winner of each match into the next round

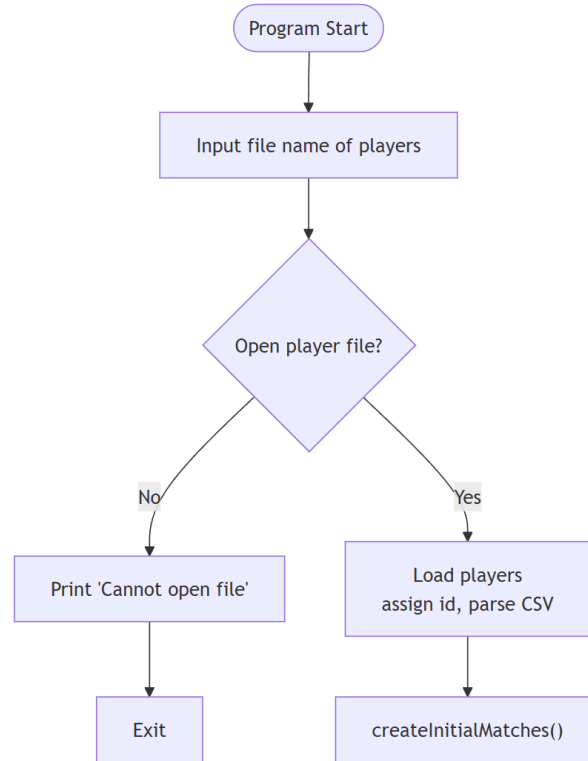
1. For each match, simulate its competition
2. Pair up the winners for the next round
3. Store all the matches into **rounds** to keep record for future use

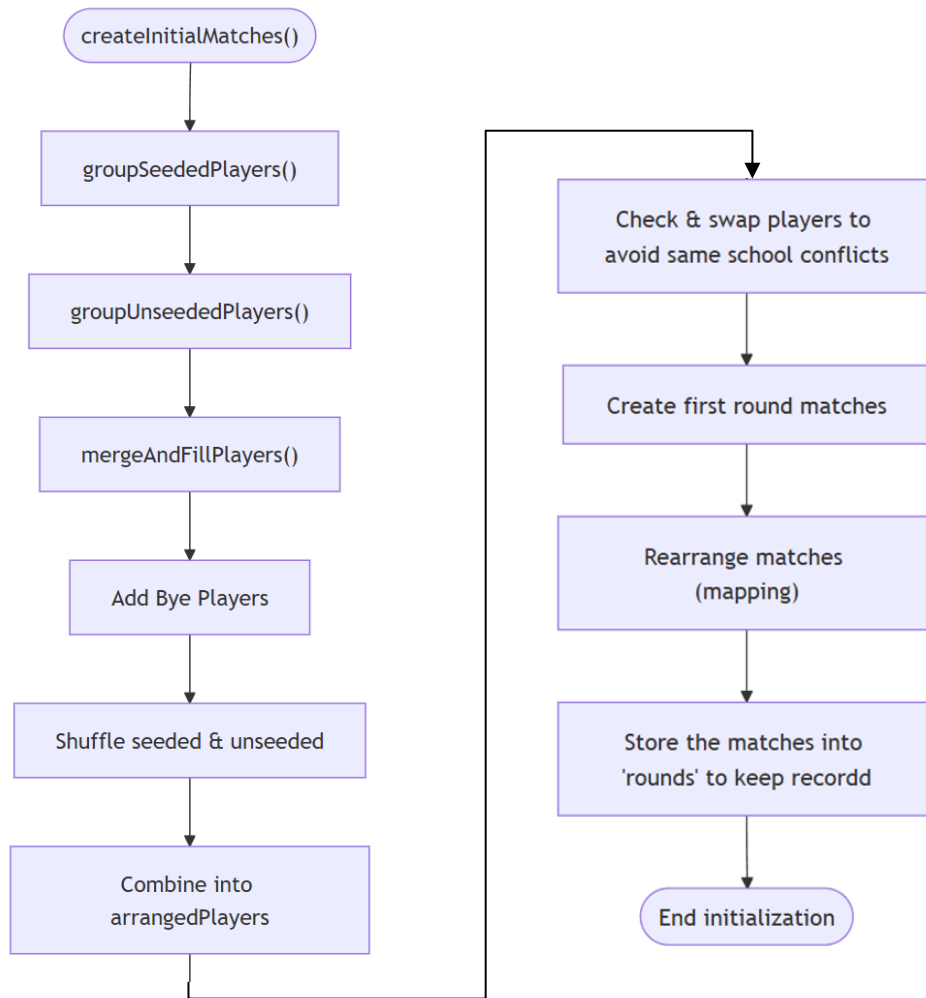
Special case:

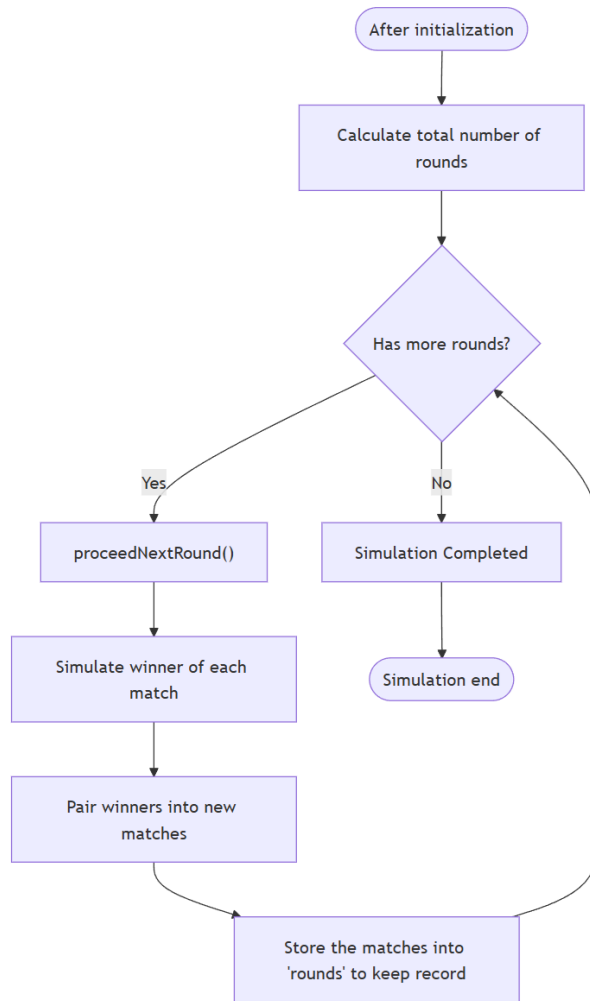
when there is 1 player left, no need to make another round (Champion)

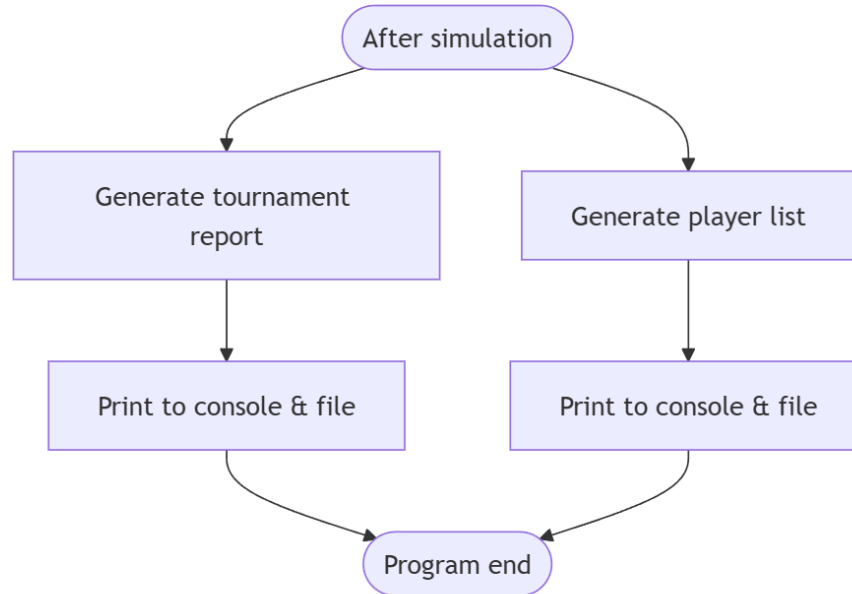
Entry Point

Flowchart of main()









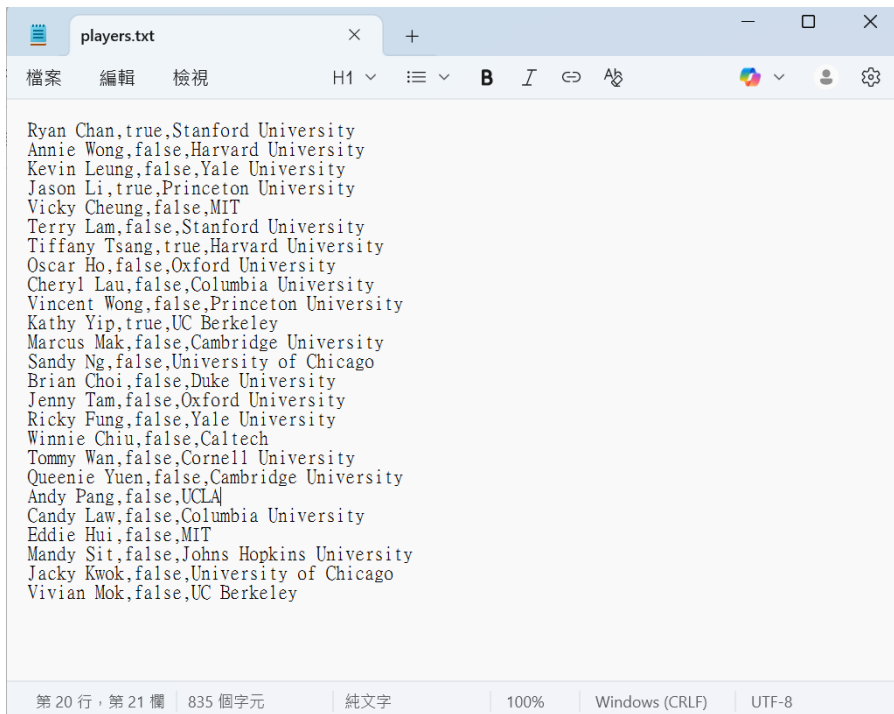
Entry point of the program

```
354
355 int main()
356 {
357     // generate random seed
358     srand(time(nullptr));
359
360     // create tournament object and load in players
361     Tournament tournament;
362
363     string playerLoadFileName;
364     ifstream playerLoadFile;
365
366     // validate player file
367     do {
368         cout<<"File name of players to load in: ";
369         cin>>playerLoadFileName;
370         playerLoadFile.open(playerLoadFileName);
371     }while(!PlayerLoader::isValidFile(playerLoadFile));
372
373
374     PlayerLoader::loadPlayers(tournament.players, playerLoadFile);
375
376
377     // run the tournament
378     tournament.simulateTournament();
379
380
381     // create output files
382     ofstream tourReportFile("tournamentReport.txt");
383     ofstream playerListFile("playerList.txt");
384
385     // print the tournament reports to .txt files and console
386     tournament.printTo(tournament.generatePlayerList(), cout);
387     tournament.printTo(tournament.generatePlayerList(), playerListFile);
388
389     tournament.printTo(tournament.generateTournamentReport(), cout);
390     tournament.printTo(tournament.generateTournamentReport(), tourReportFile);
391
392     tourReportFile.close();
393     playerListFile.close();
394
```

1. Generate a random seed
2. It asks the user to input the file name containing players data
3. Load all the players into the tournament
4. It simulates the tournament and output the results into both .txt files and console

How to use the program

Preparation – a list of player info



A screenshot of a text editor window titled "players.txt". The window displays a list of 20 players, each with their name, a boolean seed value, and their school name, separated by commas. The text is as follows:

```
Ryan Chan,true,Stanford University
Annie Wong,false,Harvard University
Kevin Leung,false,Yale University
Jason Li,true,Princeton University
Vicky Cheung,false,MIT
Terry Lam,false,Stanford University
Tiffany Tsang,true,Harvard University
Oscar Ho,false,Oxford University
Cheryl Lau,false,Columbia University
Vincent Wong,false,Princeton University
Kathy Yip,true,UC Berkeley
Marcus Mak,false,Cambridge University
Sandy Ng,false,University of Chicago
Brian Choi,false,Duke University
Jenny Tam,false,Oxford University
Ricky Fung,false,Yale University
Winnie Chiu,false,Caltech
Tommy Wan,false,Cornell University
Queenie Yuen,false,Cambridge University
Andy Pang,false,UCLA
Candy Law,false,Columbia University
Eddie Hui,false,MIT
Mandy Sit,false,Johns Hopkins University
Jacky Kwok,false,University of Chicago
Vivian Mok,false,UC Berkeley
```

The status bar at the bottom of the window shows: 第 20 行, 第 21 欄 | 835 個字元 | 純文字 | 100% | Windows (CRLF) | UTF-8

User should prepare a list of player in this format (csv):
([name],[seed (true/false)],[school name])

Input file name

```
PS C:\TEMP\C (1)> ./a.exe  
File name of players to load in: 
```

Upon executing the program, user will be greeted by a line that asks them to input the name of the .txt file containing the player info that they created beforehand

Input file name

```
PS C:\TEMP\C (1)> g++ main.cpp
PS C:\TEMP\C (1)> ./a.exe
File name of players to load in: players.txt
25 players successfully loaded!
```

User shall input the name of the file they wish the players to be imported from as shown

ID	NAME	SEED	SCHOOL
1	Ryan Chan	seed	Stanford University
2	Annie Wong		Harvard University
3	Kevin Leung		Yale University
4	Jason Li	seed	Princeton University
5	Vicky Cheung		MIT
6	Terry Lam		Stanford University
7	Tiffany Tsang	seed	Harvard University
8	Oscar Ho		Oxford University
9	Cheryl Lau		Columbia University
10	Vincent Wong		Princeton University
11	Kathy Yip	seed	UC Berkeley
12	Marcus Mak		Cambridge University
13	Sandy Ng		University of Chicago
14	Brian Choi		Duke University
15	Jenny Tam		Oxford University
16	Ricky Fung		Yale University
17	Winnie Chiu		Caltech
18	Tommy Wan		Cornell University
19	Queenie Yuen		Cambridge University
20	Andy Pang		UCLA
21	Candy Law		Columbia University
22	Eddie Hui		MIT
23	Mandy Sit		Johns Hopkins University
24	Jacky Kwok		University of Chicago
25	Vivian Mok		UC Berkeley

A player list would be generated

```
=====Round of 16=====
Tiffany Tsang* vs Bye
Match winner: Tiffany Tsang*

Eddie Hui vs Marcus Mak
Match winner: Eddie Hui

Oscar Ho vs Cheryl Lau
Match winner: Oscar Ho

Jacky Kwok vs Bye
Match winner: Jacky Kwok

Annie Wong vs Bye
Match winner: Annie Wong

Brian Choi vs Vivian Mok
Match winner: Vivian Mok

Winnie Chiu vs Kevin Leung
Match winner: Winnie Chiu

Jason Li* vs Bye
Match winner: Jason Li*

Ryan Chan* vs Bye
Match winner: Ryan Chan*

Jenny Tam vs Queenie Yuen
Match winner: Jenny Tam

Mandy Sit vs Tommy Wan
Match winner: Tommy Wan

Sandy Ng vs Andy Pang
Match winner: Andy Pang

Terry Lam vs Bye
Match winner: Terry Lam

Ricky Fung vs Candy Law
Match winner: Ricky Fung

Vincent Wong vs Vicky Cheung
Match winner: Vicky Cheung

Kathy Yip* vs Bye
Match winner: Kathy Yip*
```

```
=====Round of 8=====
Tiffany Tsang* vs Eddie Hui
Match winner: Tiffany Tsang*

Oscar Ho vs Jacky Kwok
Match winner: Oscar Ho

Annie Wong vs Vivian Mok
Match winner: Annie Wong

Winnie Chiu vs Jason Li*
Match winner: Jason Li*

Ryan Chan* vs Jenny Tam
Match winner: Ryan Chan*

Tommy Wan vs Andy Pang
Match winner: Tommy Wan

Terry Lam vs Ricky Fung
Match winner: Ricky Fung

Vicky Cheung vs Kathy Yip*
Match winner: Kathy Yip*
```

```
=====Quarterfinals=====
Tiffany Tsang* vs Oscar Ho
Match winner: Tiffany Tsang*

Annie Wong vs Jason Li*
Match winner: Jason Li*

Ryan Chan* vs Tommy Wan
Match winner: Ryan Chan*

Ricky Fung vs Kathy Yip*
Match winner: Ricky Fung

=====Semifinals=====
Tiffany Tsang* vs Jason Li*
Match winner: Jason Li*

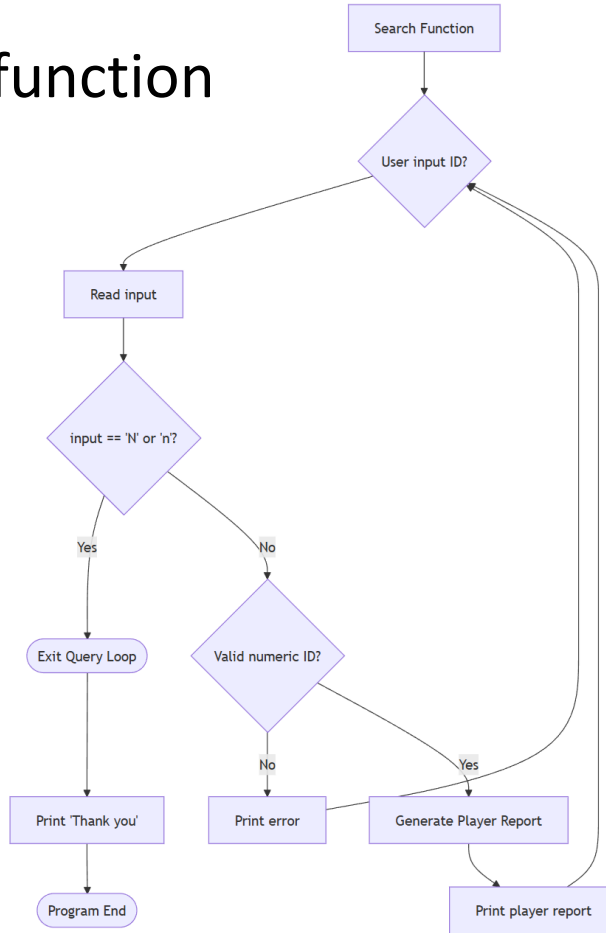
Ryan Chan* vs Ricky Fung
Match winner: Ryan Chan*

=====Final=====
Jason Li* vs Ryan Chan*
Match winner: Ryan Chan*

Final champion: Ryan Chan* (Stanford University)
```

Tournament output (console)

Optional: player query function



Optional: function for enquiring player record

```
369
370 // check player records
371 string userInput;
372 while(true) {
373     cout<<"Enter N or n to exit"<<endl;
374     cout<<"Enter player ID to see their records: ";
375     cin>>userInput;
376     cout<<endl;
377
378     if(userInput == "N" || userInput == "n") break;
379
380     int id;
381     try{
382         id = stoi(userInput);
383     }catch(const std::invalid_argument & e){
384         cout<<"Invalid input: not a number"<<endl;
385         continue;
386     }catch(const std::out_of_range & e){
387         cout<<"Invalid input: number too large / too small"<<endl;
388         continue;
389     }
390
391     // validate inputs
392     if (id > tournament.players.size() || id <= 0) {
393         cout<<"Invalid input: out of range"<<endl;
394         continue;
395     }
396
397     tournament.printTo(tournament.generatePlayerReport(id), cout);
398 };
399
```

User can input a player ID to see the player's record
To exit the program, user can enter "N" or "n"

```
Enter N or n to exit
Enter player ID to see their records: 4

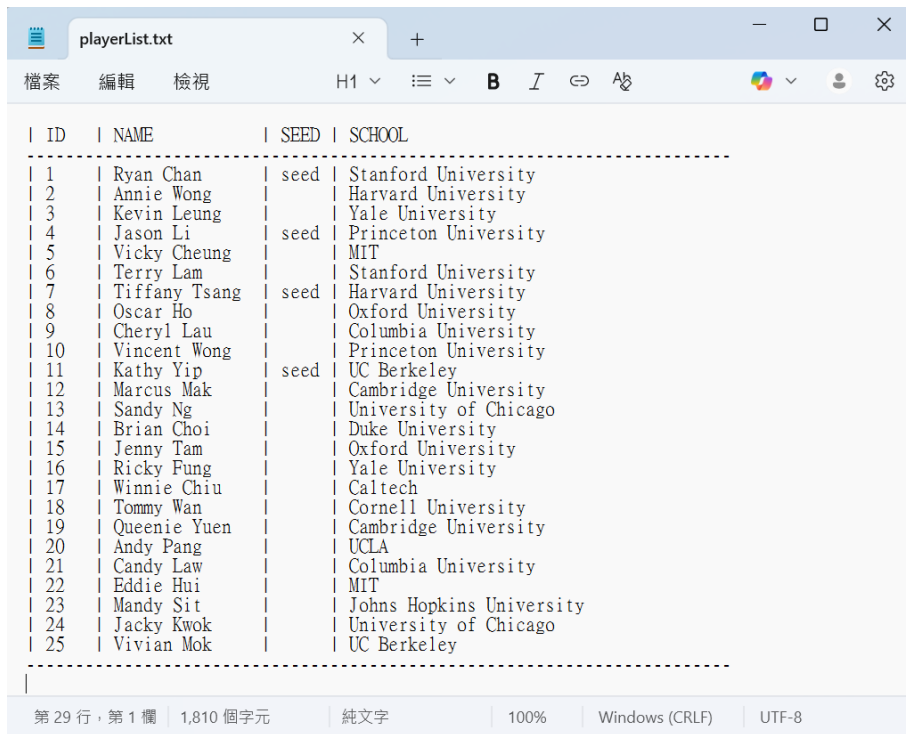
Player searching for: Jason Li
-----
Round of 16
Jason Li* vs Bye
Match winner: Jason Li*
-----
Round of 8
Winnie Chiu vs Jason Li*
Match winner: Jason Li*
-----
Quarterfinals
Annie Wong vs Jason Li*
Match winner: Jason Li*
-----
Semifinals
Tiffany Tsang* vs Jason Li*
Match winner: Jason Li*
-----
Final
Jason Li* vs Ryan Chan*
Match winner: Ryan Chan*
-----
Total matches: 5

Enter N or n to exit
Enter player ID to see their records: 
```

```
Enter N or n to exit
Enter player ID to see their records: n

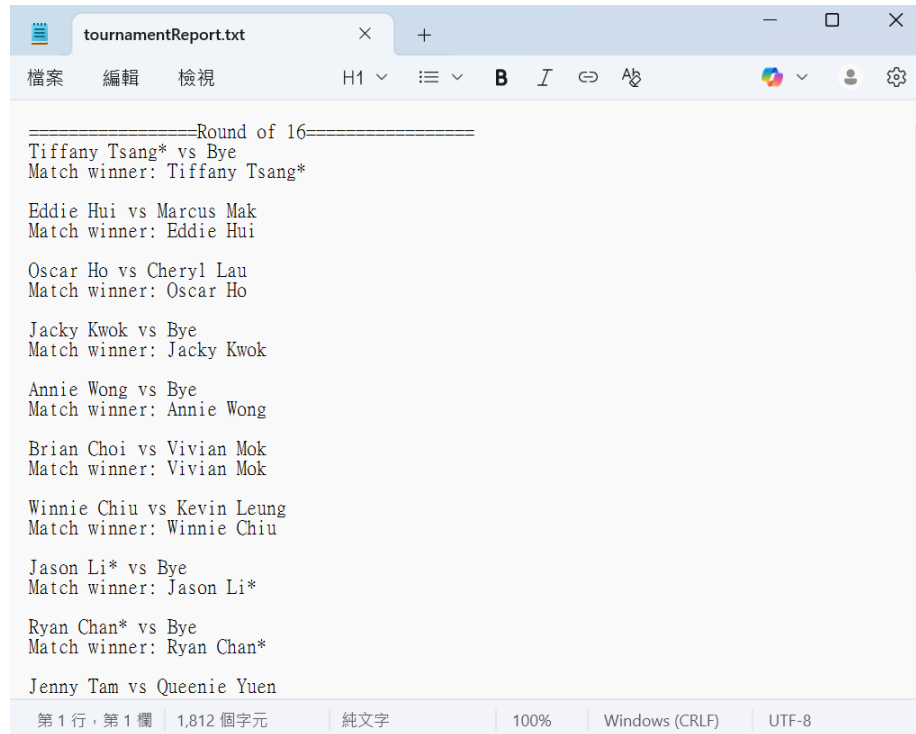
Thank you for using this program!
PS C:\TEMP\C (1)>
```

File output



ID	NAME	SEED	SCHOOL
1	Ryan Chan	seed	Stanford University
2	Annie Wong		Harvard University
3	Kevin Leung		Yale University
4	Jason Li	seed	Princeton University
5	Vicky Cheung		MIT
6	Terry Lam		Stanford University
7	Tiffany Tsang	seed	Harvard University
8	Oscar Ho		Oxford University
9	Cheryl Lau		Columbia University
10	Vincent Wong		Princeton University
11	Kathy Yip	seed	UC Berkeley
12	Marcus Mak		Cambridge University
13	Sandy Ng		University of Chicago
14	Brian Choi		Duke University
15	Jenny Tam		Oxford University
16	Ricky Fung		Yale University
17	Winnie Chiu		Caltech
18	Tommy Wan		Cornell University
19	Queenie Yuen		Cambridge University
20	Andy Pang		UCLA
21	Candy Law		Columbia University
22	Eddie Hui		MIT
23	Mandy Sit		Johns Hopkins University
24	Jacky Kwok		University of Chicago
25	Vivian Mok		UC Berkeley

Player list



Match	Winner
Tiffany Tsang* vs Bye	Tiffany Tsang*
Eddie Hui vs Marcus Mak	Eddie Hui
Oscar Ho vs Cheryl Lau	Oscar Ho
Jacky Kwok vs Bye	Jacky Kwok
Annie Wong vs Bye	Annie Wong
Brian Choi vs Vivian Mok	Vivian Mok
Winnie Chiu vs Kevin Leung	Winnie Chiu
Jason Li* vs Bye	Jason Li*
Ryan Chan* vs Bye	Ryan Chan*
Jenny Tam vs Queenie Yuen	

Tournament report

Task 2

- Data validations used in the program
- User feedbacks and improvements
- Further developments

Data validations

- In order to prevent runtime errors and to enhance user experience, this program include some validation checks
- Some examples will be shown below

Validating the number of players entering

```
168
169 int getNumberOfMatches(int playerListSize)
170 {
171     //special cases
172     if (playerListSize == 0)
173         return 0;
174     if (playerListSize == 1)
175         return 1;
176
177     return pow(2, ceil(log(playerListSize) / log(2))) / 2;
178 };
179
180
```

- All player objects will be stored in a vector before all simulations of the tournament
- Since this method uses logarithms to find the number of matchups that are needed to be created for any number of competitors, the input values should be validated to handle special cases:
 - when there are no players in list, there should be 0 matchups
 - when there are 1 player in list, there should be exactly 1 matchup (player automatically wins)

Validating the input file

```
62
63 static bool isValidFile(ifstream &file){
64     if (!file.is_open())
65     {
66         cout << "Cannot open file, please re-enter" << endl;
67         file.close();
68         return false;
69     }
70     if (file.peek() == EOF)
71     {
72         cout << "File is empty, please re-enter" << endl;
73         file.close();
74         return false;
75     }
76     return true;
77 }
78
```

In case the file cannot be opened properly (no such file exists), the user will be asked to re-enter the file name again as shown

```
PS C:\TEMP\C (1)> g++ main.cpp
PS C:\TEMP\C (1)> .\a.exe
File name of players to load in: thisfiledoesntexist.txt
Cannot open file, please re-enter
File name of players to load in: 
```

In case the file is empty (does not contain any player info), the user will also be asked to re-enter file name as shown

```
PS C:\TEMP\C (1)> g++ main.cpp
PS C:\TEMP\C (1)> .\a.exe
File name of players to load in: thisfileisempty.txt
File is empty, please re-enter
File name of players to load in: 
```

Validating user inputs for enquiry function

```

390
391 // check player records
392 string userInput;
393 while(true) {
394     cout<<"Enter N or n to exit"<<endl;
395     cout<<"Enter player ID to see their records: ";
396     cin>>userInput;
397     cout<<endl;
398
399     if(userInput == "N" || userInput == "n") break;
400
401     int id;
402     try{
403         id = stoi(userInput);
404     }catch(const std::invalid_argument & e){
405         cout<<"Invalid input: not a number"<<endl;
406         continue;
407     }catch(const std::out_of_range & e){
408         cout<<"Invalid input: number too large / too small"<<endl;
409         continue;
410     }
411
412     // validate inputs
413     if (id > tournament.players.size() || id <= 0) {
414         cout<<"Invalid input: out of range"<<endl;
415         continue;
416     }
417
418     tournament.printTo(tournament.generatePlayerReport(id), cout);
419 };

```

In case the user inputs other than a number, or the number doesn't fit in the int data type, the user will receive an error message and will be asked to input again

```
Enter player ID to see their records: abc
Invalid input: not a number
Enter N or n to exit
Enter player ID to see their records: |
```

[illegible]

This part validates that the player ID entered by the user exists

```
Enter player ID to see their records: 150
Invalid input: out of range
Enter N or n to exit
Enter player ID to see their records: 
```

User feedbacks

Problem: console output is hard to follow

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	100	101	102	103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127	128	129	130	131	132	133	134	135	136	137	138	139	140	141	142	143	144	145	146	147	148	149	150	151	152	153	154	155	156	157	158	159	160	161	162	163	164	165	166	167	168	169	170	171	172	173	174	175	176	177	178	179	180	181	182	183	184	185	186	187	188	189	190	191	192	193	194	195	196	197	198	199	200	201	202	203	204	205	206	207	208	209	210	211	212	213	214	215	216	217	218	219	220	221	222	223	224	225	226	227	228	229	230	231	232	233	234	235	236	237	238	239	240	241	242	243	244	245	246	247	248	249	250	251	252	253	254	255	256	257	258	259	260	261	262	263	264	265	266	267	268	269	270	271	272	273	274	275	276	277	278	279	280	281	282	283	284	285	286	287	288	289	290	291	292	293	294	295	296	297	298	299	300	301	302	303	304	305	306	307	308	309	310	311	312	313	314	315	316	317	318	319	320	321	322	323	324	325	326	327	328	329	330	331	332	333	334	335	336	337	338	339	340	341	342	343	344	345	346	347	348	349	350	351	352	353	354	355	356	357	358	359	360	361	362	363	364	365	366	367	368	369	370	371	372	373	374	375	376	377	378	379	380	381	382	383	384	385	386	387	388	389	390	391	392	393	394	395	396	397	398	399	400	401	402	403	404	405	406	407	408	409	410	411	412	413	414	415	416	417	418	419	420	421	422	423	424	425	426	427	428	429	430	431	432	433	434	435	436	437	438	439	440	441	442	443	444	445	446	447	448	449	450	451	452	453	454	455	456	457	458	459	460	461	462	463	464	465	466	467	468	469	470	471	472	473	474	475	476	477	478	479	480	481	482	483	484	485	486	487	488	489	490	491	492	493	494	495	496	497	498	499	500	501	502	503	504	505	506	507	508	509	510	511	512	513	514	515	516	517	518	519	520	521	522	523	524
---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

- Users reported that the tournament results are too long and difficult to follow
- The entire player list and all tournament rounds are printed to the console at once immediately after loading in the players, producing a wall of text that is hard to read and quickly scrolls out of view

Solution: add a pause between printing each round

```
136
137 void printTournamentResultsToConsole(){
138
139     for (auto &report : tournamentResults){
140
141         // pause the output
142         cout << "Press enter to proceed...";
143         cin.ignore(numeric_limits<streamsize>::max(), '\n');
144         cin.get();
145
146         cout<<report.str();
147
148     }
149
150 }
151
```

Modify the console output so that the player list and each tournament round are shown one at a time, pausing after each section and waiting for the user to press Enter before continuing, instead of printing everything at once

Demonstration

```
PS C:\TEMP\C (1)> g++ main.cpp
PS C:\TEMP\C (1)> .\a.exe
File name of players to load in: players.txt
25 players successfully loaded!

| ID | NAME | SEED | SCHOOL |
-----|-----|-----|-----|
| 1 | yan Chan | seed | Stanford University |
| 2 | Annie Wong | | Harvard University |
| 3 | Kevin Leung | | Yale University |
| 4 | Jason Li | seed | Princeton University |
| 5 | Vicky Cheung | | MIT |
| 6 | Terry Lam | | Stanford University |
| 7 | Tiffany Tsang | seed | Harvard University |
| 8 | Oscar Ho | | Oxford University |
| 9 | Cheryl Lau | | Columbia University |
| 10 | Vincent Wong | | Princeton University |
| 11 | Kathy Yip | seed | UC Berkeley |
| 12 | Marcus Mak | | Cambridge University |
| 13 | Sandy Ng | | University of Chicago |
| 14 | Brian Choi | | Duke University |
| 15 | Jenny Tam | | Oxford University |
| 16 | Ricky Fung | | Yale University |
| 17 | Winnie Chiu | | Caltech |
| 18 | Tommy Wan | | Cornell University |
| 19 | Queenie Yuen | | Cambridge University |
| 20 | Andy Pang | | UCLA |
| 21 | Candy Law | | Columbia University |
| 22 | Eddie Hui | | MIT |
| 23 | Mandy Sit | | Johns Hopkins University |
| 24 | Jacky Kwok | | University of Chicago |
| 25 | Vivian Mok | | UC Berkeley |

-----|-----|-----|-----|
Press enter to proceed...
```

pauses

```
22 | Eddie Hui | MIT |
23 | Mandy Sit | Johns Hopkins University |
24 | Jacky Kwok | University of Chicago |
25 | Vivian Mok | UC Berkeley |
-----|-----|-----|-----|
Press enter to proceed...
=====Round of 16=====
Kathy Yip* vs Bye
Match winner: Kathy Yip*

Tommy Wan vs Jacky Kwok
Match winner: Jacky Kwok

Ricky Fung vs Jenny Tam
Match winner: Ricky Fung

Terry Lam vs Bye
Match winner: Terry Lam

Brian Choi vs Bye
Match winner: Brian Choi

Candy Law vs Vincent Wong
Match winner: Vincent Wong

Vivian Mok vs Kevin Leung
Match winner: Kevin Leung

Jason Li* vs Bye
Match winner: Jason Li*

Tiffany Tsang* vs Bye
Match winner: Tiffany Tsang*

Cheryl Lau vs Queenie Yuen
Match winner: Queenie Yuen

Mandy Sit vs Eddie Hui
Match winner: Mandy Sit

Sandy Ng vs Vicky Cheung
Match winner: Vicky Cheung

Oscar Ho vs Bye
Match winner: Oscar Ho

Marcus Mak vs Winnie Chiu
Match winner: Marcus Mak

Andy Pang vs Annie Wong
Match winner: Annie Wong

yan Chan* vs Bye
Match winner: yan Chan*

Press enter to proceed...
```

pauses

```
Press enter to proceed...

=====Round of 8=====
Kathy Yip* vs Brian Choi
Match winner: Brian Choi

Andy Pang vs Tommy Wan
Match winner: Tommy Wan

Sandy Ng vs Kevin Leung
Match winner: Kevin Leung

Annie Wong vs yan Chan*
Match winner: Annie Wong

Jason Li* vs Terry Lam
Match winner: Jason Li*

Winnie Chiu vs Vincent Wong
Match winner: Winnie Chiu

Eddie Hui vs Vivian Mok
Match winner: Vivian Mok

Marcus Mak vs Tiffany Tsang*
Match winner: Tiffany Tsang*

Press enter to proceed...
```

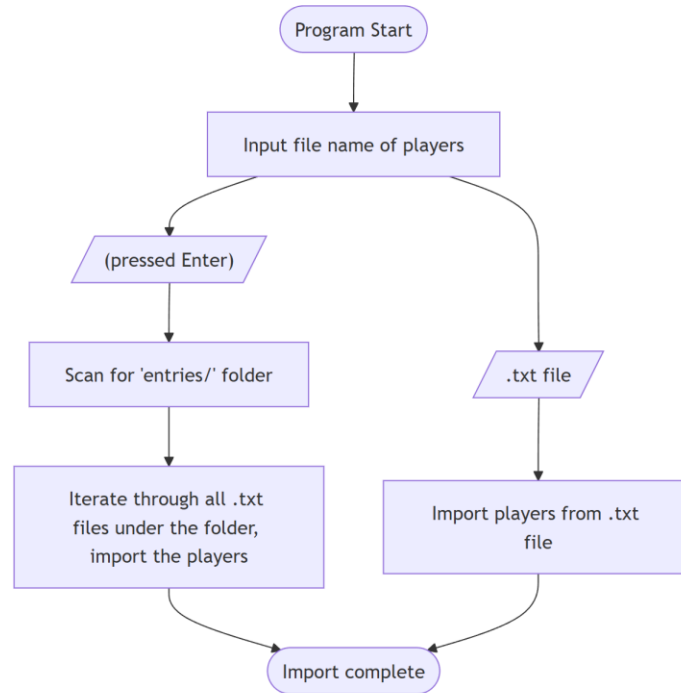
pauses

Further development

Idea: Importing players by school (1 file per school)

- Instead of importing all players from one single file (where entries from different schools are mixed and hard to manage), the program now scans for a designated folder (“entries/”) and load players from individual .txt files
- Advantages of this approach:
 - Easier to manage: each school has its own clearly name file (e.g. MIT.txt)
 - User friendliness: each school can create and submit their own small file, and there’s no need to merge to one big file before using
 - Better readability: users can easily see which schools are participating just by looking at the folder, and easily keep track of the participants of a specific school
 - Enhanced user experience overall

Flowchart of the data entry



Implementing in the program

```
80 static void loadPlayersFromFolder(vector<Player> &players, string folderName){
81     string dir = "../" + folderName;
82
83     vector<filesystem::path> txt_files;
84
85     // read from folder
86     for (const auto& entry : filesystem::directory_iterator(dir)) {
87         if (entry.is_regular_file() && entry.path().extension() == ".txt") {
88             txt_files.push_back(entry.path());
89         }
90     }
91
92     string line;
93     // assign unique id to each player
94     int playerCount = 1;
95
96     // keep track of the number of seeded players
97     int seededPlayerCount = 0;
98
99     for (auto &fileName : txt_files){
100         int schoolPlayerCount = 0;
101         ifstream playerFile(fileName.string());
102
103         if (playerFile.peek() == EOF)
104         {
105             cout<<"ERROR: School '"
106                 <<fileName.stem().string()
107                 <<"' file is empty."<<endl;
108             cout<<"Exiting program..."<<endl;
109             exit(0);
110         }
111
112         while (getline(playerFile, line))
113         {
114             vector<string> playerData;
115
116             stringstream ss(line);
117             while (ss.good())
118             {
119                 string substr;
120                 getline(ss, substr, ',');
121                 playerData.push_back(substr);
122             }
123
124             bool seed = (playerData[1] == "true");
125             seededPlayerCount += seed;
126
127             Player player(playerData[0], seed, fileName.stem().string());
128             player.id = playerCount++;
129             players.push_back(player);
130         }
131     }
132 }
```

The mentioned feature could be done by adding a method that scan for a designated folder ("entries/") containing one .txt file per school.

The code iterates through all .txt files under the folder, assigns unique IDs, and load the players into the tournament

Validating the files

```
// check if school has more than 2 players
if (schoolPlayerCount > 2){
    cout<<"ERROR: School '"<<fileName.stem().string()
        <<"' has more than 2 players ("<<schoolPlayerCount
        <<" found). Each school is allowed at most 2 players."<<endl;
    cout<<"Exiting program..."<<endl;
    exit(0);
}

// check if more than 4 seeds
if (seededPlayerCount > 4){
    cout<<"ERROR: More than 4 seeded players found."<<endl;
    cout<<"Exiting program..."<<endl;
    exit(0);
}
}
```

The program validates each school's entry file. If a file contains more than 2 players, an error message specifying the school name and number of entries will be displayed. The program will be exited immediately.

```
PS C:\TEMP\C (1)> .\a.exe
File name of players to load in (Press enter to load from 'entries' folder):
ERROR: School 'MIT' has more than 2 players (4 found). Each school is allowed at most 2 players.
Exiting program...
PS C:\TEMP\C (1)> 
```

If the number of seeded players exceed 2, there will also be an according error message. The program will then be exited.

```
PS C:\TEMP\C (1)> .\a.exe
File name of players to load in (Press enter to load from 'entries' folder):
ERROR: More than 4 seeded players found.
Exiting program...
PS C:\TEMP\C (1)> 
```

Empty file case

```
if (playerFile.peek() == EOF)
{
    cout<<"ERROR: School '"
        <<fileName.stem().string()
        <<"' file is empty."<<endl;
    cout<<"Exiting program..."<<endl;
    exit(0);
}
```

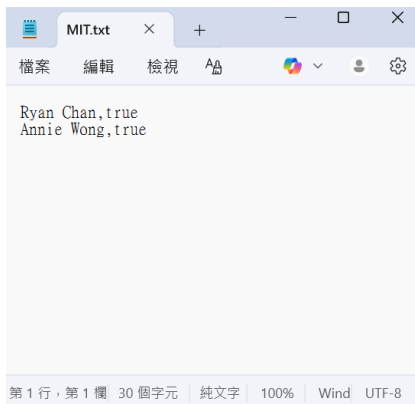
Empty files will also be validated

```
PS C:\TEMP\C (1)> .\a.exe
File name of players to load in (Press enter to load from 'entries' folder):
ERROR: School 'EmptySchool' file is empty.
Exiting program...
PS C:\TEMP\C (1)> 
```

How to use – preparation

名稱	修改日期	類型	大小
a.exe	11/12/2025 21:35	應用程式	452 KB
entries	11/12/2025 21:54	檔案資料夾	

名稱	修改日期	類型	大小
MIT.txt	11/12/2025 21:46	TXT 檔案	1 KB
Oxford University.txt	11/12/2025 21:46	TXT 檔案	1 KB
Stanford University.txt	11/12/2025 21:46	TXT 檔案	1 KB



1. User should create a folder named 'entries' in program directory
2. Create one .txt file for each participating school. File name should be school name
3. Inside each .txt file, data should be in the format:
((name],[seed (true/false)])

*There is no need to enter school name for each individual player record, since the program automatically takes the file name as school name

How to use – entering the program

```
PS C:\TEMP\C (1)> .\a.exe  
File name of players to load in (Press enter to load from 'entries' folder):
```



```
PS C:\TEMP\C (1)> .\a.exe  
File name of players to load in (Press enter to load from 'entries' folder):  
6 players successfully loaded!
```

ID	NAME	SEED	SCHOOL
1	Ryan Chan	seed	MIT
2	Annie Wong		MIT
3	Kevin Leung	seed	Oxford University
4	Jason Li	seed	Oxford University
5	Oscar Ho		Stanford University
6	Cheryl Lau		Stanford University

```
Press enter to proceed...
```

Upon entering the program, user will be asked to enter a file name the same as before. They can input the name of the single big file containing all player info and use the program exactly as before

To use the new feature, user can simply press Enter without typing anything. The program will automatically load players from all .txt files located in 'entries' folder

Idea: Adding colors to console output

- Currently, all the console output appears in plain white text, which is difficult to read. Adding different font colors would greatly improves readability
- Advantages:
 - Critical information (validation failures, winners and losers, etc.) can be spotted instantly
 - Better aesthetics: Using different font color for different types of information enhances the aesthetic of the interface

Implementing in the program

```
1  #define RESET    "\033[0m"
2  #define RED      "\033[31m"
3  #define GREEN    "\033[32m"
4  #define YELLOW   "\033[38;5;220m"
5  #define BLUE     "\033[34m"
6  #define MAGENTA  "\033[35m"
7  #define CYAN     "\033[36m"
8  #define WHITE    "\033[37m"
9  #define BOLD     "\033[1m"
10
```

At the top of the code, add these definitions of different colors

Then, font colors can be changed easily by adding the predefined color macros before cout

```
if (!file.is_open())
{
    cout << RED << "Cannot open file, please re-enter" << RESET << endl;
    file.close();
    return false;
}
```

```
cout<<GREEN<<players.size()<<" players successfully loaded!"<<RESET<<endl;
cout<<endl;
```


Demonstration

```
PS C:\TEMP\C (1)> .\a.exe
File name of players to load in (Press enter to load from 'entries' folder):
6 players successfully loaded!

| ID | NAME | SEED | SCHOOL |
|----|-----|-----|-----|
| 1  | Ryan Chan | seed | MIT |
| 2  | Annie Wong |  | MIT |
| 3  | Kevin Leung | seed | Oxford University |
| 4  | Jason Li | seed | Oxford University |
| 5  | Oscar Ho |  | Stanford University |
| 6  | Cheryl Lau |  | Stanford University |
|----|-----|-----|-----|

Press enter to proceed...

=====Quarterfinals=====
Ryan Chan* vs Bye
Match winner: Ryan Chan*

Jason Li* vs Cheryl Lau
Match winner: Jason Li*

Annie Wong vs Oscar Ho
Match winner: Oscar Ho

Kevin Leung* vs Bye
Match winner: Kevin Leung*

Press enter to proceed...

=====Semifinals=====
Ryan Chan* vs Jason Li*
Match winner: Jason Li*

Oscar Ho vs Kevin Leung*
Match winner: Kevin Leung*

Press enter to proceed...

=====Final=====
Jason Li* vs Kevin Leung*
Match winner: Kevin Leung*

Final champion: Kevin Leung* (Oxford University)
```

Success messages are shown in green

```
PS C:\TEMP\C (1)> .\a.exe
File name of players to load in (Press enter to load from 'entries' folder): nosuchfile.txt
Cannot open file, please re-enter
File name of players to load in (Press enter to load from 'entries' folder):
ERROR: School 'MIT' has more than 2 players (4 found). Each school is allowed at most 2 players.
Exiting program...
PS C:\TEMP\C (1)>
```

Error messages are shown in red

Important information are in different colors

Idea: Replace hard-coded Mapping class with dynamic algorithm

- Currently, the function which is responsible for separating seeded players as far as possible is hard-coded, supporting only up to 16 matches (32 players)

```
30 class Mapping
31 {
32 public:
33     static vector<int> getMapping(int matchCount){
34         if (matchCount == 4) return {0,2,3,1};
35         if (matchCount == 8) return {0,4,6,2,3,7,5,1};
36         if (matchCount == 16) return {0,8,12,4,6,14,10,2,3,11,15,7,5,13,9,1};
37         else return {};
38     }
39 };
```

- While it is possible to expand the support by adding more hard-coded cases, this is not ideal
- By implementing a dynamic algorithm, no code changes for larger tournaments will ever be needed, with zero risks of manual errors when hard-coding those cases. Any sizes of tournament could be supported

End